

Day 2: Async JavaScript & the Event Loop

Intern: Penumuru Madhu Sudhan Reddy

Topic: Asynchronous Programming, Event Loop, Promises, and Concurrency

Target: Beginner-Friendly & Practical

Overview: JavaScript runs in browsers and Node.js on a **single thread**. If a network request took 5 seconds to load and froze the thread, the entire website would stop responding. Asynchronous programming is how JavaScript executes long-running tasks in the background without freezing your application.

1. Synchronous vs Asynchronous Code

- **Synchronous Code:** Runs line by line. Each line must finish before the next line starts.
- **Asynchronous Code:** Starts a task (like a timer, file read, or network call) in the background and moves to the next line immediately. When the background task finishes, its callback runs.

DEMONSTRATING NON-BLOCKING ASYNCHRONOUS CODE

```
console.log("1. Program starts");

// Schedules a callback after 2000 milliseconds (2 seconds)
setTimeout(() => {
  console.log("2. Timer finished (runs 2 seconds later)");
}, 2000);

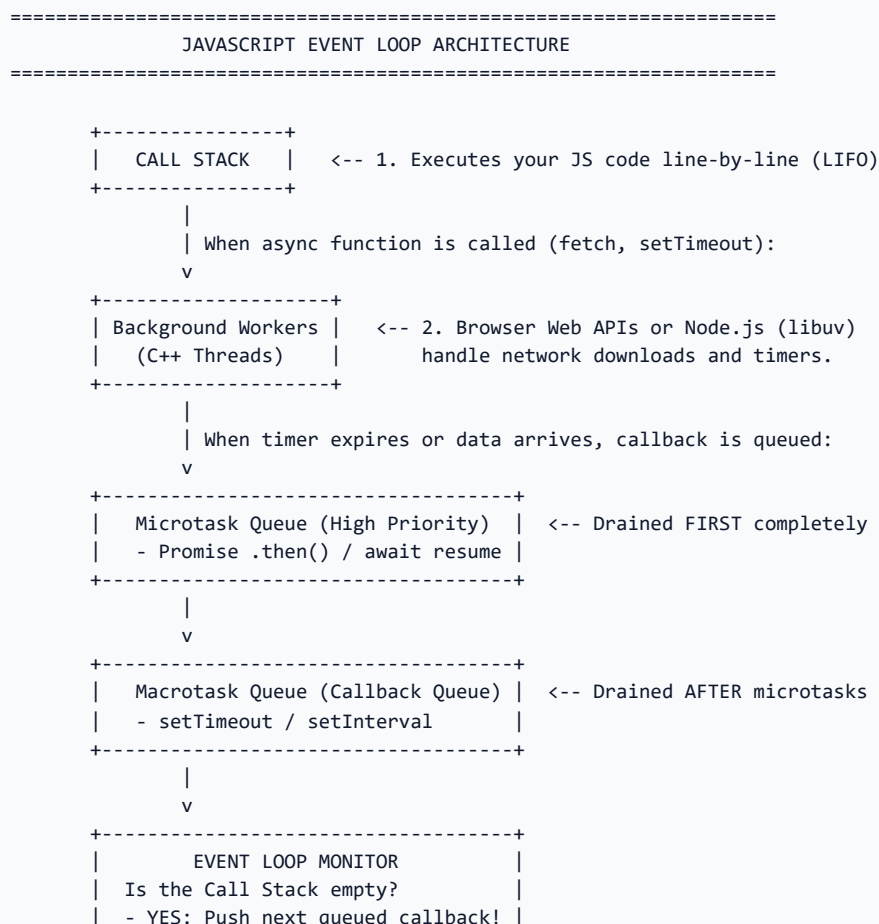
console.log("3. Program continues immediately!");
```

Terminal Output:

```
1. Program starts
3. Program continues immediately!
2. Timer finished (runs 2 seconds later)
```

2. How the Event Loop Works Under the Hood

The JavaScript runtime has 4 key parts that work together to coordinate async code:



```
| - NO: Wait for stack to finish. |  
+-----+
```

Priority Rule: Microtasks vs Macrotasks

- **Microtasks:** Promise callbacks (`.then()`, `await` continuations). They have **top priority** and drain completely before any timer runs.
- **Macrotasks:** `setTimeout` and `setInterval` callbacks. They run only when the Microtask queue is empty.

3. Evolution: Callbacks → Promises → async/await

Step 1: Callbacks (The Old Way)

Early JavaScript passed functions as arguments. When you had multiple sequential steps, it created hard-to-read nested code ("Callback Hell"):

```
getUser(1, (user) => {  
  getProjects(user.id, (projects) => {  
    getTowers(projects[0].id, (towers) => {  
      console.log("Towers:", towers);  
    });  
  });  
});
```

Step 2: Promises (The Modern Foundation)

A **Promise** is an object that represents a future result. It has 3 states:

- **Pending:** Operation is still running in background.
- **Fulfilled:** Operation succeeded → access result with `.then(data => ...)`.
- **Rejected:** Operation failed → catch error with `.catch(err => ...)`.

CREATING AND CONSUMING A PROMISE

```
function checkSiteSafety(score) {  
  return new Promise((resolve, reject) => {  
    if (score >= 70) {  
      resolve("Safety Certified: Score " + score);  
    } else {  
      reject(new Error("Safety Audit Failed: Score below 70"));  
    }  
  });  
}  
  
checkSiteSafety(85)  
  .then(message => console.log("Success:", message))  
  .catch(error => console.log("Error:", error.message));
```

Terminal Output:

```
Success: Safety Certified: Score 85
```

Step 3: async / await (The Modern Standard)

`async/await` is built on Promises. It lets you write asynchronous code that reads cleanly like normal sequential code:

USING ASYNC / AWAIT WITH TRY/CATCH

```
async function runSafetyCheck() {  
  try {  
    const result = await checkSiteSafety(92);  
    console.log("Inspection Result:", result);  
  } catch(err) {  
    console.log("Inspection Failed:", err.message);  
  }  
}  
runSafetyCheck();
```

Terminal Output:

```
Inspection Result: Safety Certified: Score 92
```

4. The `fetch()` API for Network Requests

`fetch()` is the standard built-in way to make HTTP requests in modern JavaScript and Node.js.

Why fetch() Requires Two Awaits:

1. `const response = await fetch(url);` → Waits for the HTTP response headers (status 200, Content-Type).
2. `const data = await response.json();` → Asynchronously streams and parses the JSON response body.

COMPLETE FETCH() EXAMPLE WITH ERROR HANDLING

```
async function getTodo() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");
    if (!res.ok) {
      throw new Error(`HTTP error: ${res.status}`);
    }
    const data = await res.json();
    console.log("Todo Title:", data.title);
  } catch (err) {
    console.log("Fetch Error:", err.message);
  }
}
getTodo();
```

Terminal Output:

```
Todo Title: delectus aut autem
```

5. Promise Combinators: Parallel vs Sequential Execution

When you need to make multiple requests, choosing between sequential and parallel execution has a massive impact on performance:

Method	How It Works	Error Behavior	When to Use at NirmanIQ
<code>Promise.all([p1, p2, p3])</code>	Runs all in parallel concurrently.	Fails fast: If even one fails, the entire call rejects immediately.	Loading project view where Project, Tower, and User details are all required.
<code>Promise.allSettled([p1, p2])</code>	Runs all to completion.	Never rejects! Returns array of { status, value/reason }.	Bulk uploading 20 site inspection photos (find out which passed and which failed).
<code>Promise.race([p1, p2])</code>	First promise to finish wins.	Resolves or rejects based on the fastest promise.	Setting request timeouts (race API call against a 5-second timer promise).

Performance Benchmark: Sequential (3s) vs Parallel (1s)

RUNNING 3 APIS IN PARALLEL WITH PROMISE.ALL()

```
const sleep = ms => new Promise(r => setTimeout(r, ms));

async function fetchTask(name, delay) {
  await sleep(delay);
  return `${name} loaded`;
}

async function runBenchmark() {
  console.time("Parallel Total Time");
  // All 3 start at the exact same millisecond:
  const results = await Promise.all([
    fetchTask("Projects", 600),
    fetchTask("Towers", 600),
    fetchTask("Workers", 600)
  ]);
  console.timeEnd("Parallel Total Time"); // Only ~600ms total!
  console.log("Results:", results);
}
runBenchmark();
```

Terminal Output:

```
Parallel Total Time: 604ms
Results: [ 'Projects loaded', 'Towers loaded', 'Workers loaded' ]
```

6. Practical Design Patterns Built on Day 2

Pattern 1: Exponential Backoff Retry (Network Resilience)

Construction mobile connections drop packets constantly. Exponential backoff doubles the wait time between retries (500ms → 1000ms → 2000ms):

DAY2/EXERCISES/EXERCISE-2-RETRY-FETCH.JS

```
async function fetchWithRetry(url, maxRetries = 3) {
  let delay = 500;
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP error ${res.status}`);
      return await res.json();
    } catch(err) {
      if (attempt === maxRetries) throw err;
      console.log(`Attempt ${attempt} failed. Retrying in ${delay}ms...`);
      await new Promise(r => setTimeout(r, delay));
      delay = delay * 2; // double delay each time
    }
  }
}
```

Pattern 2: Worker Pool Queue (Concurrency Limit of 3)

Converting 100 site videos simultaneously crashes server memory. A worker pool runs exactly 3 parallel workers that pull tasks from a shared list until all are completed:

DAY2/EXERCISES/EXERCISE-3-TASK-QUEUE.JS

```
async function processQueue(tasks, limit = 3) {
  const results = [];
  let index = 0;

  async function worker() {
    while (index < tasks.length) {
      const current = index++;
      results[current] = await processVideo(tasks[current]);
    }
  }

  // start 3 workers in parallel:
  const workers = [];
  for (let i = 0; i < limit; i++) workers.push(worker());
  await Promise.all(workers);
  return results;
}
```